

JavaScript Fundamentals

Notes with explanations, based on concepts covered in app.js

1. Objects

An object groups related data (properties) and behavior (methods) together under one variable. Properties are written as **key: value** pairs, separated by commas.

```
var car = {
  name: 'mira',
  cc: 660,
  brand: 'daihatsu',
  color: 'black',
  model: 2026,
  startEngine: function () {
    console.log("starting engine...");
  }
}

console.log(car.name);    // "mira"
```

You access a property with dot notation (**car.name**) or bracket notation (**car["name"]**). A function stored as a property, like **startEngine**, is called a **method**.

Nested arrays and objects

Object values are not limited to strings and numbers — they can be arrays or even other objects, letting you model more complex data.

```
var car = {
  name: 'mira',
  feature: ['comfort', 'power-window', 'power steering'],
  engineComponent: {
    piston: 50
  }
}

console.log(car.feature[car.feature.length - 1]); // "power steering"
car.cc = 1000;    // adding/updating a property
console.log(car.cc); // 1000
```

Objects created with **var/let** are mutable by default: you can add new properties or change existing ones after creation, even though the variable itself was not reassigned.

2. Arrays of Objects and Looping

A very common pattern is storing several related objects inside an array, then looping over the array to process each one.

```

var std_1 = { name: 'hamza', id: 1 };
var std_2 = { name: 'bilal', id: 2 };
var std_3 = { name: 'ali', id: 3 };

var stds = [std_1, std_2, std_3];

for (var i = 0; i < stds.length; i++) {
  console.log(stds[i].name);
}
// hamza
// bilal
// ali

```

A classic **for** loop uses a counter (**i**) to index into the array until it reaches **stds.length**.

3. Constructor Functions and Prototypes

Instead of writing the same object shape by hand every time, a **constructor function** acts as a template. Calling it with **new** creates a fresh object and binds **this** to it.

```

function CreateObj(name, id) {
  this.name = name;
  this.id = id;
}

CreateObj.prototype.greet = function () {
  console.log("hello from " + this.name);
};

var std_1 = new CreateObj("kaif", 12);
var std_2 = new CreateObj("ahmed", 3);
var std_3 = new CreateObj("bilal", 15);

std_1.greet(); // "hello from kaif"
std_2.greet(); // "hello from ahmed"
std_3.greet(); // "hello from bilal"

```

Adding **greet** to **CreateObj.prototype** means every instance shares the same single function in memory, instead of each object getting its own copy. This is the mechanism ES6 **class** syntax is built on top of.

4. let, const, and Block Scope

let declares a variable that can be reassigned. **const** declares a variable whose **binding** cannot be reassigned after it is set.

```

let a = 20;
console.log(a); // 20

const b = 20;
console.log(b); // 20
// b = 30;      // TypeError: Assignment to constant variable.

```

Block scoping

let and **const** are scoped to the nearest **{ }** block (an **if**, a loop, etc.), unlike **var**, which ignores blocks and is only scoped to the function or the global scope.

```
let num = 9;

if (num > 4) {
  let a = 3;
  console.log(a); // 3, only visible inside this block
}

if (true) {
  const a = 5;
  console.log(a); // 5
}

a = 4;          // this refers to a DIFFERENT, outer/global "a"
console.log(a); // 4
```

Each **if** block above has its own independent **a**. The final **a = 4** is assigning to a variable declared elsewhere (outside these blocks), which is why it does not clash with the block-scoped ones.

5. Arrow Functions

Arrow functions (`=>`) are a shorter syntax for writing functions. When the body is a single expression, the braces and **return** keyword can be dropped entirely — this is called an **implicit return**.

```
const test = () => {
  console.log("hello");
};
test(); // "hello"

const print = a => a + "b";
console.log(print(1)); // "1b"
```

print takes one argument and implicitly returns **a + "b"**, without needing curly braces or a **return** statement.

6. Destructuring

Array destructuring

Destructuring unpacks values from an array (or object) into separate named variables in one line, based on position.

```
const arr = [1, 2, 3, 4, 5];
const [a, b, c, d, e] = arr;

console.log(c, d); // 3 4
```

Object destructuring

For objects, the variable names must match the property names you want to pull out.

```
const obj = {
  name: "daniyal",
  age: 26
};

const { name, age } = obj;
console.log(name); // "daniyal"
console.log(age); // 26
```

7. The Spread Operator (...)

The spread operator `...` expands an iterable (array, string, etc.) into its individual elements. It is used both to pass array items as separate arguments and to build new arrays from existing ones.

```
const arr = ["kaif", "ahsan", "bilal", "hamza"];
console.log(...arr);
// kaif ahsan bilal hamza (each as a separate argument to console.log)

const nums = [1, 2, 3];
const nums2 = [...nums, 4, 5, 6];
console.log(nums2); // [1, 2, 3, 4, 5, 6]
```

nums2 is a brand-new array: the original **nums** is copied in, then 4, 5, 6 are added after it. This is a common way to add to an array without mutating the original.

8. The for...of Loop

for...of loops directly over the **values** of an iterable (arrays, strings, Maps, Sets), without needing an index counter.

```
const fruits = ["apple", "banana", "cherry"];

for (const fruit of fruits) {
  console.log(fruit);
}
// apple
// banana
// cherry
```

9. Array.map() and Array.filter()

map() — transform every element

map() creates a **new array** by applying a function to every element of the original array. The original array is left unchanged.

```
const even = [2, 4, 6, 8, 10, 12];

const odd = even.map((v) => {
  return v + 1;
});

console.log(odd); // [3, 5, 7, 9, 11, 13]
```

filter() — keep only matching elements

filter() also returns a new array, but it only keeps elements for which the callback returns **true**.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const evens = numbers.filter((v) => {
  return v % 2 === 0; // true for even numbers
});

console.log(evens); // [2, 4, 6, 8, 10]
```

10. Default Parameters

A parameter can be given a default value, which is used only when the caller does not supply an argument for it (or passes **undefined**).

```
function test(a, b = 5) {
  console.log(a + b);
}

test(1, 2); // 3    -> b is 2 (explicitly passed)
test(1);    // 6    -> b defaults to 5
```

11. Rest Parameters (...)

Inside a function's parameter list, **...** collects all remaining arguments into a single real array, called the **rest parameter**. This is the same **...** syntax as spread, but used in the opposite direction: gathering instead of expanding.

```
function sum(a, b, ...c) {
  let result = 0;

  for (let i = 0; i < c.length; i++) {
    result = result + c[i];
  }

  console.log(result + a + b);
}
```

```
sum(1, 2, 3, 4, 5, 6); // a=1, b=2, c=[3,4,5,6] -> 21
```

The same logic can be written more concisely with **reduce()**, which folds an array down to a single value:

```
function sum(a, b, ...c) {  
  let result = c.reduce((p, c) => p + c, 0);  
  console.log(result + a + b);  
}
```

```
sum(1, 2, 3, 4, 5, 6); // 21
```

reduce((accumulator, current) => ..., initialValue) repeatedly combines array elements into one running total, starting from the initial value (**0** here).

Another rest parameter example: product

```
function product(a, b, ...c) {  
  let result = 1;  
  
  for (let i = 0; i < c.length; i++) {  
    result = result * c[i];  
  }  
  
  console.log(result * a * b);  
}
```

```
product(1, 2, 3, 4, 5, 6); // a=1, b=2, c=[3,4,5,6] -> 720
```

12. setInterval() and setTimeout()

Both schedule code to run later, but differently:

setTimeout — runs once

Runs a function **one time**, after the given delay in milliseconds.

```
setTimeout(() => {  
  console.log("hello from timeout!");  
}, 3000); // runs once, after 3 seconds
```

setInterval — runs repeatedly

Runs a function **again and again**, every N milliseconds, until it is stopped (e.g. with **clearInterval**).

```
setInterval(() => {  
  console.log("hello from interval!");  
}, 1000); // runs every 1 second, forever (until cleared)
```

Because both are asynchronous, they do not block the rest of the script — code after them keeps running immediately, and the callbacks fire later on their own schedule.